

Week 2 SDLC

AI-Assisted Coding 시대에서 시스템 분석 및 설계의 중요성에 대해 간단히 학습합니다.

Install Codex CLI / Gemini CLI / Claude Code

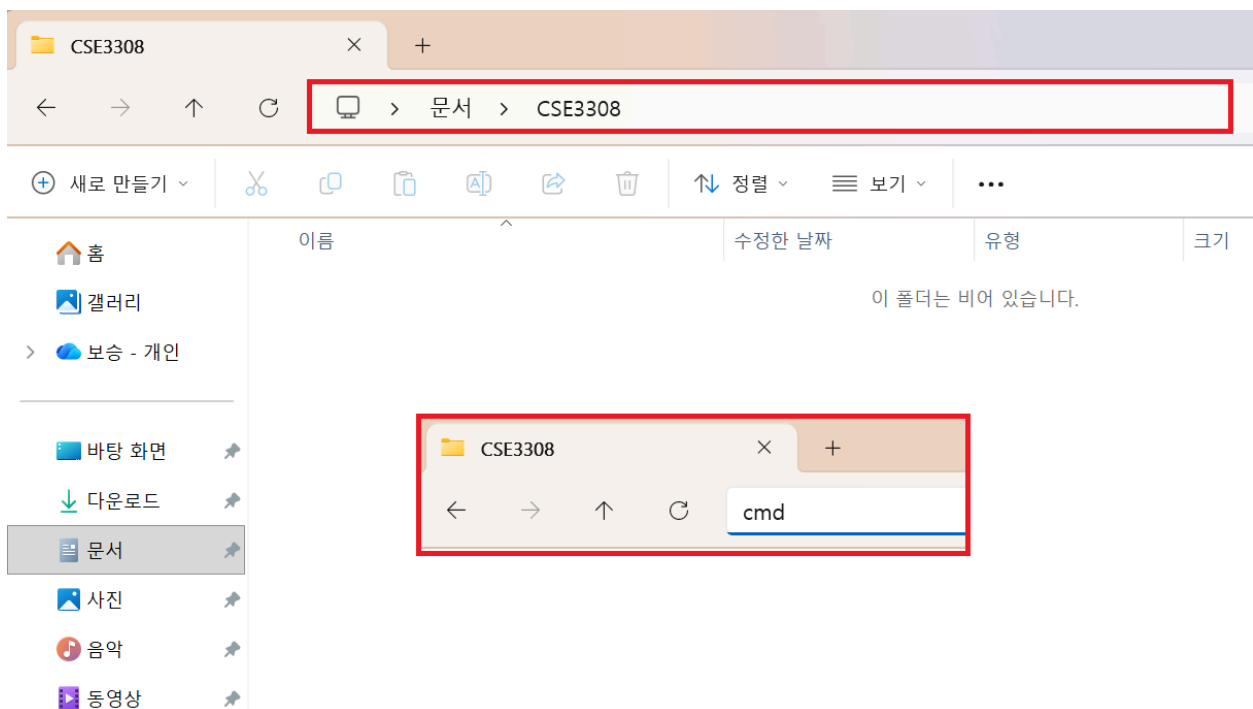
- [Node.js — Node.js® 다운로드](#)
- [Codex CLI](#)
- [Gemini CLI installation, execution, and releases | Gemini CLI](#)
- [Advanced setup - Claude Code Docs](#)

Pre-requirements

- [Git - Install](#)
- [Download Python | Python.org](#)

실습을 위해 설치된 파이썬의 버전은 3.10 이상이어야 합니다.

How to start with CLI (Example in Windows)



```
C:\Windows\system32\cmd
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\meno9\Documents\CSE3308>gemini
```

- gemini
- codex
- claude

CLI 최초 실행의 경우 Web-Auth 작업이 필요합니다.

YOLO Mode

```
\CSE3308>claude --dangerously-skip-permissions
```

- claude —dangerously-skip-permissions
- gemini —yolo
- codex —yolo

각 도구에게 무제한적 권한을 부여합니다. 예전에는 실수로 C드라이브를 날려버리는 일도 발생했습니다.

Quick Vibe Coding Example

```
> Codex, randomized quicksort algorithm을 시각화 할 수 있는 간단한 도구 하나를 만들어주세요.

• 요청하신 “랜덤화 퀵정렬 시각화”를 바로 구현할 수 있게 먼저 저장소 구조를 확인한 뒤, 브라우저에서 실행 가능한 간단한 시각화 도구를 추가하겠습니다. 먼저 현재 프로젝트 진입점을 확인해 적절한 위치에 최소 구성으로 넣겠습니다.

• Ran Get-ChildItem -Force
  ↳ (no output)

• Ran Get-ChildItem -Force -Recurse -Depth 2
  ↳ (no output)

• Ran Get-Location; Get-ChildItem
  ↳
    Path
    ... +2 lines
```

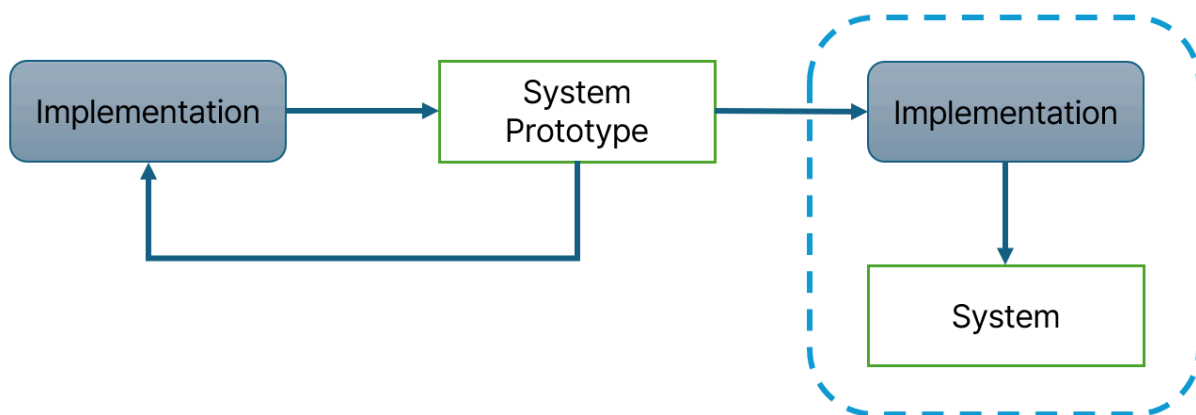


구현에 걸린 시간 : 84초

[Randomized quicksort.html](#)

- 훌륭한 Modern Style UI, 실시간 재생, 추적 가능, Log 제공, Step-by-Step 기능, 파라미터 설정

Prototyping in vite coding



기능은 잘 구현되었는데, 몇가지 문제가 있습니다. [1] 해당 도구는 정렬 알고리즘에 대해서는 잘 이해하고 있지만, Randomized Quick Sorting의 정확한 알고리즘에 대해서는 잘 모르는 User를 위한 도구를 목표로 합니다. 따라서, 추가 여백을 하나 만들어 좀 더 상세한 Log를 기술할 수 있어야 합니다. [2] 이 알고리즘이 왜 $O(n \log n)$ 의 시간 복잡도를 갖는지 시각적으로 추적할 방법이 필요합니다.



[Randomized quicksort v2.html](#)

- Incremental Requirements → 점진적으로 요구사항이 개선됨
- 이러한 상황에서의 문제점 : 비체계적 개발

- 일단 뭔가 만든다 → 결과 확인 → 분석 → 수정

Unit-Level Development vs Whole-System Development

Question


인하대학교 수강신청

휴학생 복학신청 의무 안내 (2026-1학기부터) / 수강신청 취소 여석 지연 공개 제도 안내

아이디
비밀번호
아이디/비밀번호 찾기

LOGIN

수강안내

- 수강신청안내문
- e-러닝/b-러닝 수강안내
- 공지사항
- 학번확인
- 수강신청 매뉴얼
- 수강신청포기 매뉴얼

2026 학년도 1학기 주요 수강신청 일정

일자	내용
2026. 1. 21.(수) 10:00 ~ 1. 23.(금) 16:00	우선수강신청(1차)[전체 교과목(종별 무관)][★대상자: 전체 학생] / 우선수강신청(2차): 2. 3.(화)[★대상자: 2026-1학기 다중전
2026. 2. 10.(화) 10:00 ~ 16:00	본 수강신청[전공(필수/선택), 교필, 전공기초, 교직](재수강 불가, 학년별 인원제한)[★대상자: 공과대학/소프트웨어융합대학/
2026. 2. 11.(수) 10:00 ~ 16:00	본 수강신청[전공(필수/선택), 교필, 전공기초, 교직](재수강 불가, 학년별 인원제한)[★대상자: 전체학생(공과대학/소프트웨어
2026. 2. 20.(금) 09:30 ~ 09:59	본 수강신청[전공(필수/선택), 교필, 전공기초, 교직] (2026학년도 신입생만 수강가능, 제한사항 위와 동일)
2026. 2. 20.(금) 10:00 ~ 13:00	일반 수강신청[전체 교과목(종별 무관)](재수강 불가, 수강인원 학년별 인원제한)[★대상자: 전체학생]
2026. 2. 20.(금) 14:00 ~ 16:00	일반 수강신청[전체 교과목(종별 무관)](재수강 가능, 수강인원 학년별 인원제한 없음)[★대상자: 전체학생]
2026. 2. 27.(금) 10:00 ~ 17:00	일반 수강신청[전체 교과목(종별 무관)][★대상자: 편입생, 미수강생 및 미수강복학생(0학점 신청자), 1차 폐강과목 신청자]

인하대학교의 노후화된 수강 신청 사이트를 개선하고자 한다고 생각해봅시다. 만약 이 시스템의 솔루션을 AI-Assisted로 개발하고자 한다면, 어떤 프롬프트를 작성해야합니까 ?

실제 요구사항 도출의 어려움

요구사항 명세 도출 과정

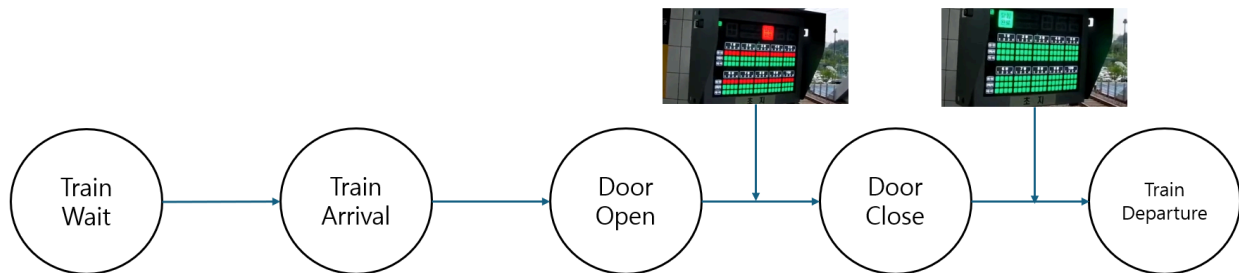


- ▶ 비즈니스 요구사항: 왜(Why)에 해당하는 정보를 나타내게 하는 것으로, 발주기관에서 프로젝트를 추진하는 배경 및 시스템을 통해 얻어지는 효과
- ▶ 사용자 요구사항: 무엇(What)에 해당하는 정보를 나타내는 것으로 시스템을 통하여 달성되는 "무엇"을 설명하며 발주기관 현업에서 수행하려는 시스템의 주요 동작기능을 설명
- ▶ 기능 요구사항: 또 다른 "무엇"에 관한 정보를 나타내는 형태로서 개발자가 개발하여야 하는 것에 대하여 기술 행위 요구사항(Behavior Requirements)이라 불리며 시스템이 반드시 수행하여야 하거나 시스템을 이용하여 발주기관에서 반드시 할 수 있어야 하는 것들에 관한 것을 언급

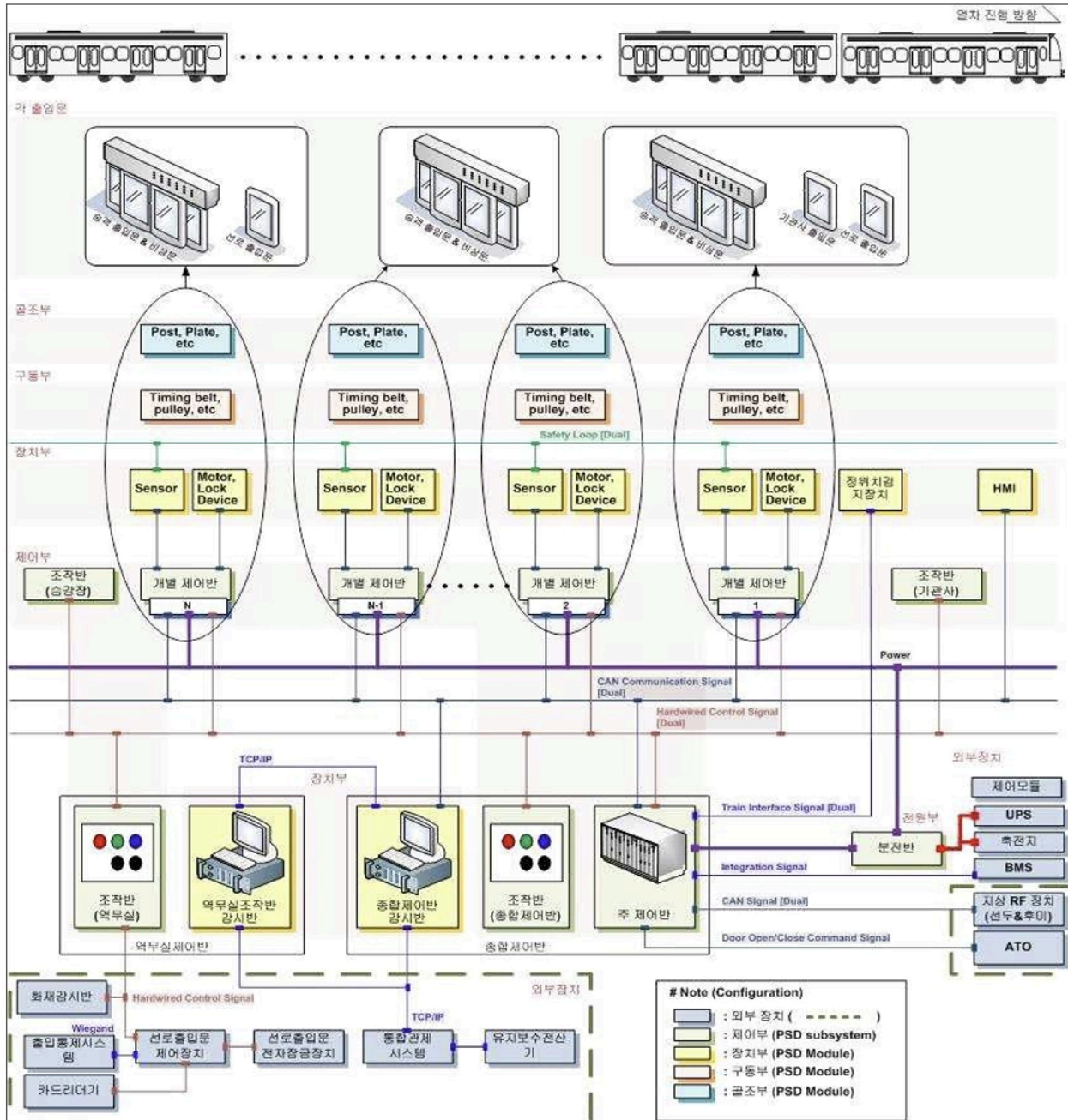
정교하고 복잡한 현실의 시스템



지하철 스크린 도어 시스템 :: 탑승객 관점



지하철 스크린 도어 시스템 :: 시스템 관점



- 단일 Quick Sort Algorithm을 시각화 하고자 하는 간단한 Tool 하나를 만드는데에도 체계적인 Requirements가 정의되지 않으면 사용자가 기대하는 결과를 내기 힘들
- 복잡한 정보 시스템 및 실세계의 Embedded 시스템들에서는 훨씬 더 큰 문제이며, 그 누구도 구체적인 문서화 및 이해관계자들의 논의 없이 제대로된 시스템을 개발할 수 없음.

Requirement Engineering

	Method / Simple Tool	Product
--	----------------------	---------

Stakeholders	User	User, Admin, Operators, ...
Expectations	Low	Extremely High
Non-Functional Requirements	None or Small	Performance, Security, Extendability, ...
Integration or Migrations	None or Small	DB, Auth, API, ...
Failure Costs	Low	High

Re-view : SDLC(Software Development Life Cycle)

- Planning: 무엇을 왜 만드는가 (System Request, Feasibility Analysis)
- Analysis: 무엇이 필요한가 (요구사항 수집, 분석 모델)
- Design: 어떻게 만드는가 (아키텍처, 인터페이스, DB 설계)
- Implementation: 실제로 만든다 (코딩, 테스트, 배포)

Re-view : System Requests

- Project Sponsor: 누가 이 프로젝트를 원하는가
- Business Need: 왜 만드는가
- Business Requirements: 무엇을 제공해야 하는가
- Business Value: 만들면 뭐가 좋은가
- Special Issues/Constraints: 제약조건은 무엇인가

Effort of Vibe Coders (2024~)

- **Vibe Coding Patterns** : Spec First, Code Later → Spec-Driven Development
- **Work like Project Manager**

1	# Taste - Tasting Notes App PRD	98	## Development Phases
2		99	
3	## Project Overview	100	### Phase 1: Foundation (Week 1)
4	Taste is a social platform for tracking and sharing tasting notes for both restaurant experiences and recipes. Users can maintain personal tasting journals, share preferences with family and friends, and discover new culinary experiences through their network.	101	- [X] Project Setup
5		102	- [X] Initialize React + TypeScript project
6	## Key Decisions & Requirements	103	- [X] Set up Firebase configuration
7		104	- [X] Configure ESLint and Prettier
8	### User Experience	105	- [X] Set up basic routing structure
9	✓ Private Notes: Users can create notes that are never shared	106	- [X] Implement basic layout components with Tailwind CSS
10	✓ Permission Levels:	107	
11	- Private (default)	108	- [X] Authentication System
12	- Friends (shared with connected users)	109	- [X] Firebase Auth integration
13	- Public (visible to all users)	110	- [X] Login/Register forms
14	✓ Rating System: 5-star rating system (1-5)	111	- [X] Password reset flow
15		112	- [X] Protected routes
16	### Data Management	113	
17	✓ Data Retention: User data retained for 1 month after account deletion	114	### Phase 2: Core Features (Weeks 2-3)
		115	- [X] Note Management
		116	- [X] Create note form with visibility options
		117	- [X] Note list view with filtering
		118	- [X] Individual note view
		119	- [X] Edit/Delete functionality

- 코딩을 시작하기 전에 PRD(Product Requirements Document)를 먼저
- PRD와 단계별 계획(Plan)을 AI와 협업하여 만들어 의도를 맞추고 작업을 분해
- SRS (Software Requirements Spec)을 작성하여 SW요구사항 명세를 LLM과 논의
- PRD와 수용 기준이 포함된 User Story를 산출

이를 통해 Why → What → How 를 달성

Case : Quick Sort Algorithm Viewer

Target User	정렬 알고리즘은 이해하지만 Randomized Quicksort를 모르는 학습자
Key Feature	상세 로그, 시간복잡도 $O(n \log n)$ 시각적 추적, 단계별 실행 필요
Constratint	브라우저 기반, 단일 HTML 파일
Acceptance Criteria	사용자가 pivot 선택의 무작위성이 최악 케이스를 방지하는 원리를 이해할 수 있어야 함

```
> Codex, randomized quicksort algorithm을 시각화 할 수 있는 간단한 도구 하나를 만들어주세요. 대상 사용자: 정렬 알고리즘
  은 이해하지만 Randomized Quicksort를 모르는 학습자
  핵심 기능: 상세 로그, 시간복잡도  $O(n \log n)$  시각적 추적, 단계별 실행
  기술 제약: 브라우저 기반, 단일 HTML 파일
  수용 기준: 사용자가 pivot 선택의 무작위성이 최악 케이스를 방지하는 원리를 이해할 수 있어야 함
```

• 브라우저에서 바로 열 수 있는 단일 HTML 시각화 도구를 만들겠습니다. 먼저 현재 작업 디렉터리 구조를 확인하고, 기존 파일이 있으면 맞춰서 최소한으로 추가하겠습니다.

```
• Running Get-ChildItem -Force
• Ran rg --files
  ↳ (no output)
• Ran Get-ChildItem -Force
  ↳ (no output)
```

• 현재 디렉터리가 비어 있어서 단일 진입 파일을 새로 만들면 됩니다. [index.html](#) 하나에 UI, 시각화, 단계 실행, 로그, 무작위 pivot 설명까지 모두 넣겠습니다.

Randomized Quicksort Visualizer

같은 입력이어도 pivot을 무작위로 뽑으면 분할 품질이 매번 달라집니다. 이 도구는 그 과정과 로그를 단계별로 보여주고, 왜 평균적으로 $O(n \log n)$ 에 머무르는지 시각적으로 추적합니다.

배열 입력

9, 4, 8, 3, 1, 2, 5, 7, 6

심표로 구분된 정수. 같은 배열로 여러 번 실행하면 pivot 선택이 어떻게 달라지는지 비교할 수 있습니다.

자동 재생 속도

520 ms / step

무작위 배열

실행 준비

다음 단계

자동 재생

처음으로

같은 입력 재실행

현재 단계

1 / 57
start

비교 횟수

0
partition에서 수행된 비교
누적

분할 깊이

0 / 0
현재 재귀 깊이 / 최대 깊이

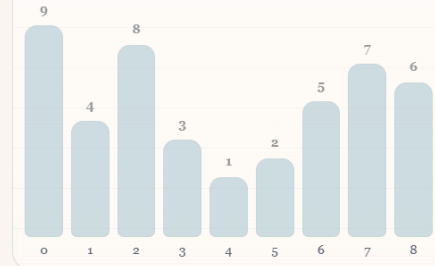
현재 level의 작업량

깊이 0에서 처리한 원소 수: 0. 균형이 좋으면 한 level당 대략 n 개를 훑습니다.

$O(n \log n)$ 추적

관측 작업량 0 / 기준 28.5. 평균적인 균형 분할이면 $O(n \log n)$ 에 가깝습니다.

Randomized Quicksort는 partition 규칙은 같고, pivot 선택만 무작위입니다. 그래서 특정 입력 패턴에 덜 취약합니다.



■ 현재 부분 배열 ■ pivot ■ 비교 중 ■ 정렬 완료

재귀 진행 상태

완료 / depth 0

상세 로그

각 단계에서 어떤 pivot이 선택되었고, 왜 그 선택이 분할 균형에 영향을 주는지 설명합니다.

1. 입력 배열 9, 4, 8, 3, 1, 2, 5, 7, 6에 대해 Randomized Quicksort를 시작합니다.

2. 깊이 0: 부분 배열 [9, 4, 8, 3, 1, 2, 5, 7, 6]에 집중합니다.

3. 깊이 0: 부분 배열 [9, 4, 8, 3, 1, 2, 5, 7, 6]에서 무작위 pivot 2를 index 5에서 선택했습니다.

4. partition을 위해 pivot 2를 오른쪽 끝으로 옮깁니다.

5. 9와 pivot 2를 비교합니다. 오른쪽 그룹에 남습니다.

6. 4와 pivot 2를 비교합니다. 오른쪽 그룹에 남습니다.

핵심 해설

실행 1: 같은 배열이라도 pivot 선택 시퀀스가 달라지면 비교 횟수와 재귀 깊이가 달라집니다. 다시 실행해 차이를 확인하세요.

현재 부분 배열

[0..8]

선택된 pivot

-

이버 브하 결과

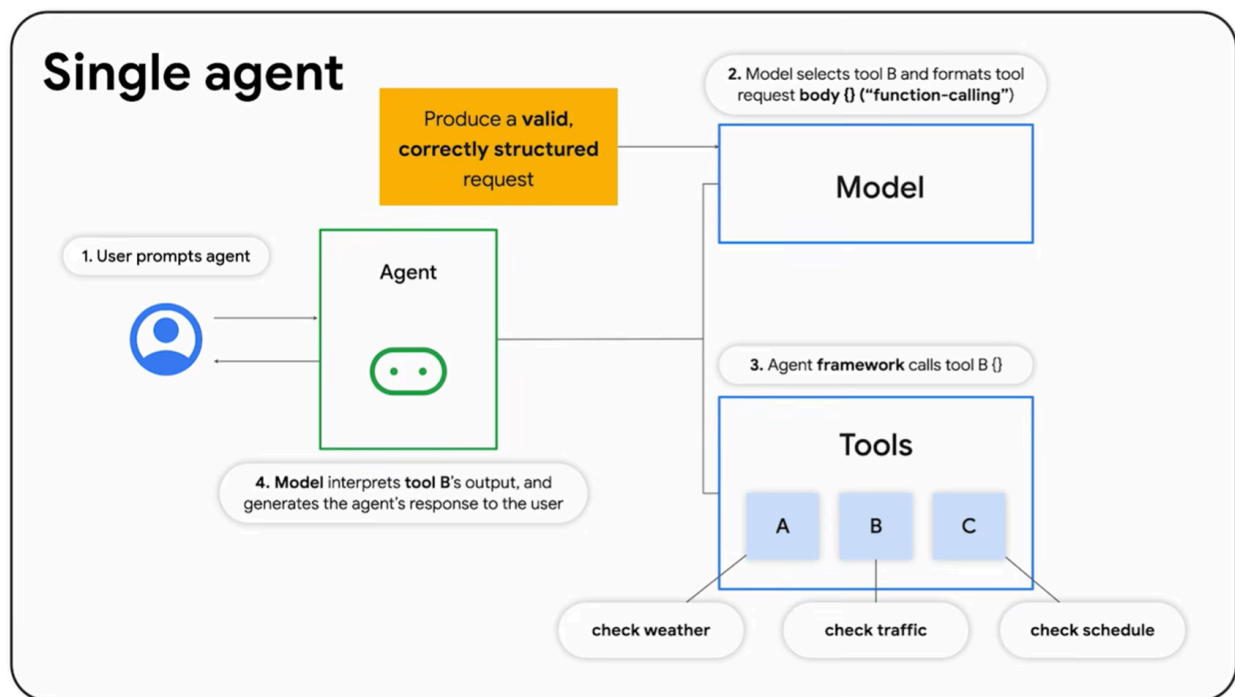
[Randomized Quick Sort v3.html](#)

- 간단한 PRD 요소를 프롬프트에 포함시킨 결과 → 월등히 향상된 결과물 생성

The Agentic Coding

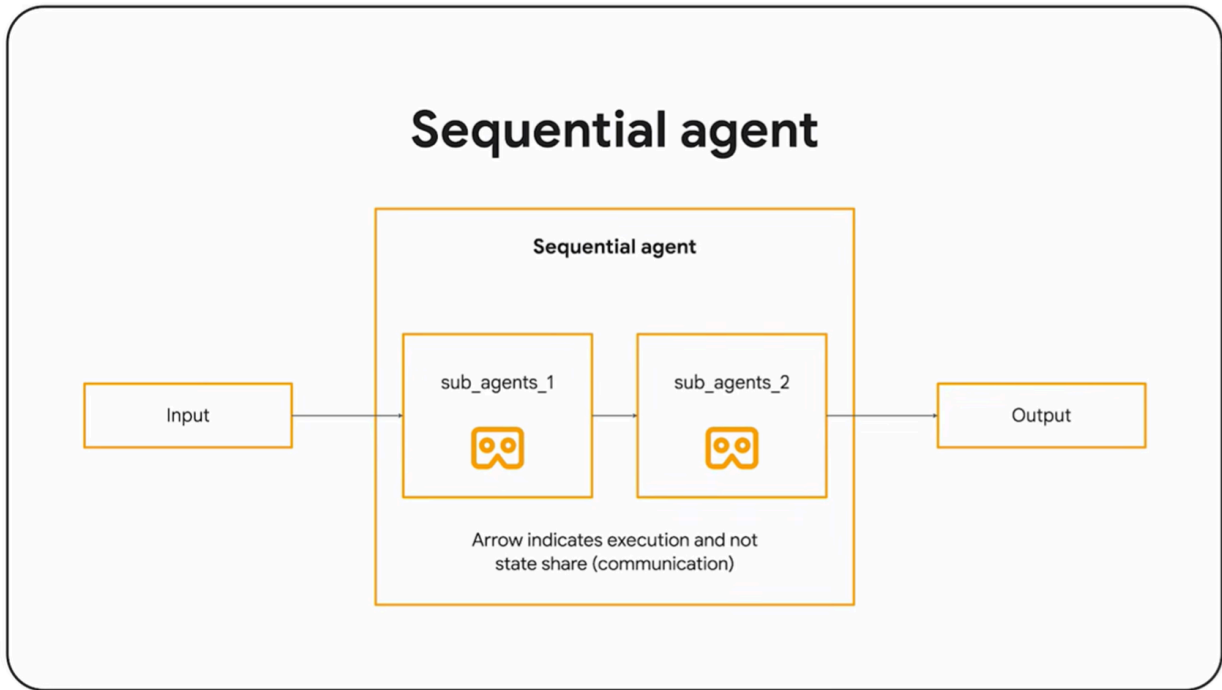
	Vibe Coding Approach	Agentic Coding Approach
접근 방식	One-Shot Generation	Description per each Agents
문서화	PRD/SRS Link or Paste (or None)	Appropriate Prompt + PRD/SRS
인간의 역할	Prompt Engineering	Orchestrator
에이전트 구조	Single Conversation	Single → Sequential → Multi-Agent
적합한 규모	Small-Size Application	Up to Large-Scale Product
SDLC 단계	Simple Loop	Co-operation with AI after SDLC Step

Single Agent



- Produce a valid, correctly structured request (사용자의 요청은 명확하고, 구조화 되어있어야 함)

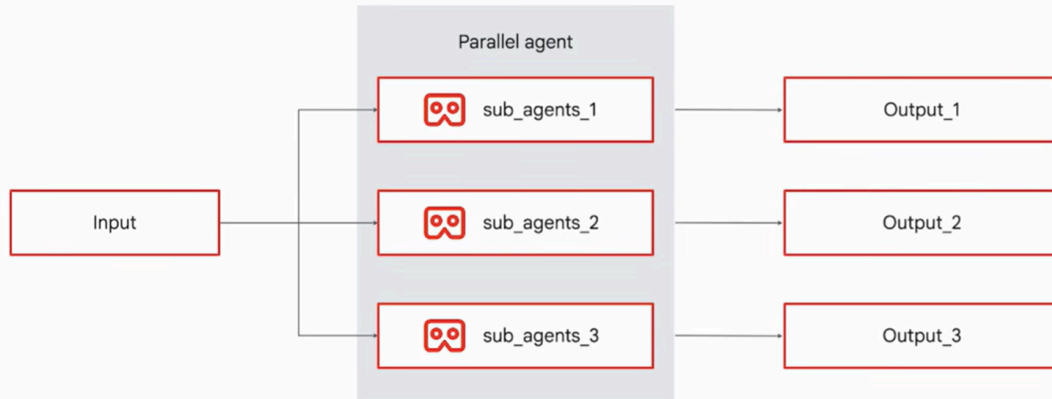
Sequential Agent



- 각 에이전트는 독립적으로 작동하되 앞 단계가 완료되어야 다음 단계가 시작
- 태스크의 순서와 의존관계를 사전에 정확히 설계해야 하며, 이것이 곧 SDLC Analysis 단계에서 수행하는 작업 분해와 동일

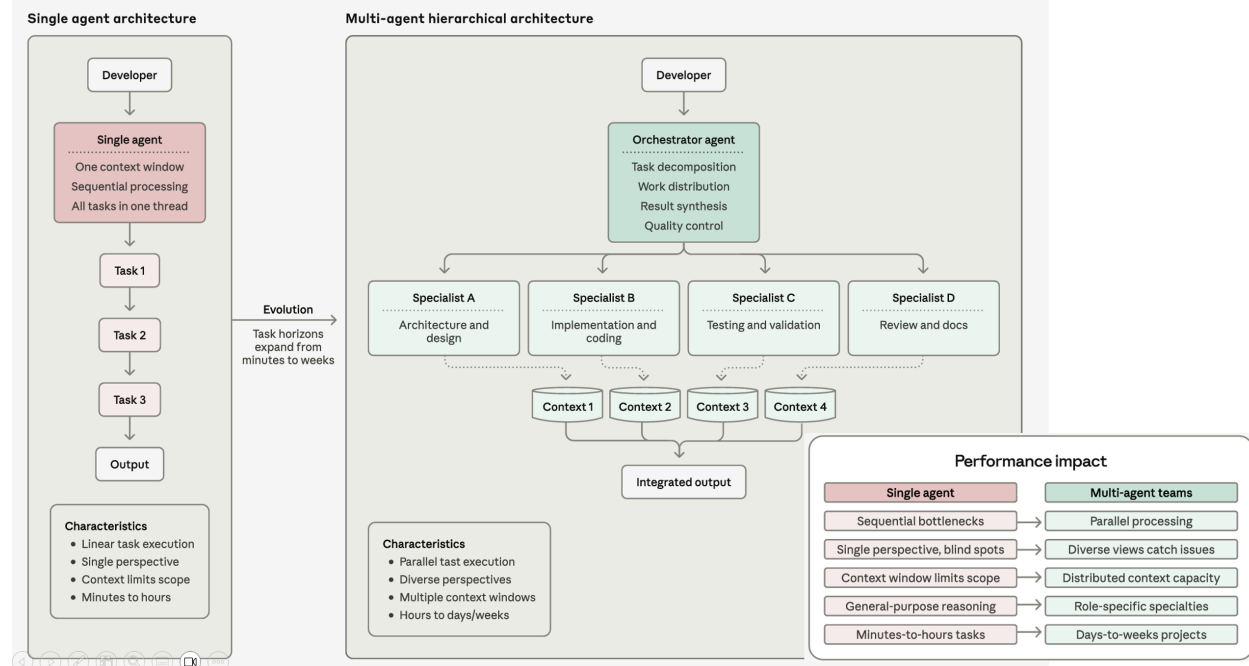
Parallel Agent

Multi-agent: Sequential with parallel (workflow)

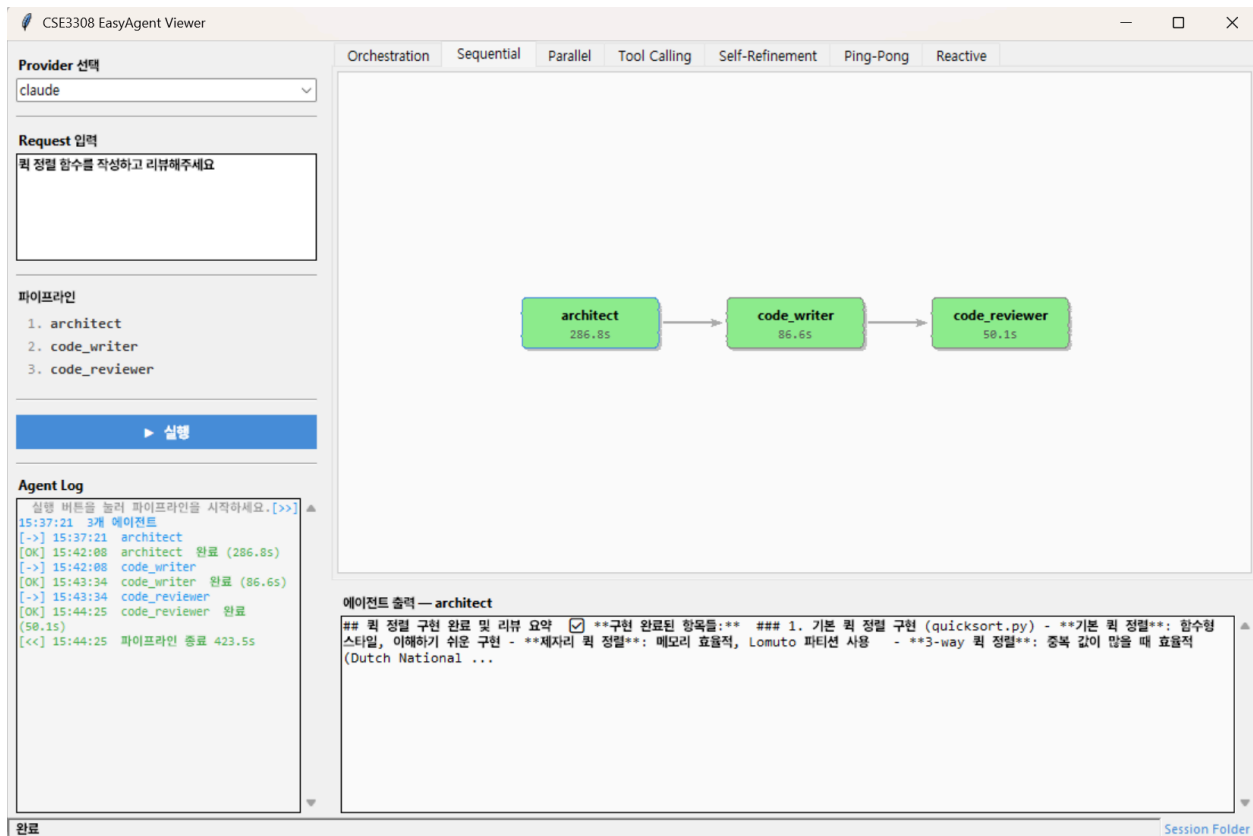


- 하나의 입력을 여러 전문화된 서브 에이전트가 동시에 처리하여 각각 독립된 결과를 산출
- 주로 Research 에서 강력한 접근 (WebSearch)

https://youtu.be/GDm_uH6VxPY?si=Hk1CQs0l3jOI_Qvc



Simple Agentic Coding Experience Tool



https://github.com/INHA-SELAB/cse3308_easy_agent

Usage

적당한 폴더에서 터미널 (cmd) 열고 :

```
git clone https://github.com/INHA-SELAB/cse3308_easy_agent.git
```

실행:

```
python main.py  
  
# python3 main.py  
# python3.10 main.py
```


- 주의 : 실용성 없는 Agentic Coding 체험용 도구입니다. 어떻게 작동하는지 살펴보신 뒤, LLM Tool
에게 작동 방식에 대해 설명해 달라고 해보세요.